

COMCAT: Leveraging Human Judgment to Improve Automatic Documentation and Summarization

Skyler Grandel, Scott Thomas Andersen, Yu Huang, and Kevin Leach

Abstract—Software maintenance constitutes a substantial portion of the total lifetime costs of software, with a significant portion attributed to code comprehension. Software comprehension is eased by documentation such as comments that summarize and explain code. We present COMCAT, an approach to automate comment generation by augmenting Large Language Models (LLMs) with expertise-guided context to target the annotation of source code with comments that improve comprehension. Our approach enables the selection of the most relevant and informative comments for a given snippet or file containing source code. We develop the COMCAT pipeline to comment C/C++ files by (1) automatically identifying suitable locations in which to place comments, (2) predicting the most helpful type of comment for each location, and (3) generating a comment based on the selected location and comment type. In a human subject evaluation, we demonstrate that COMCAT-generated comments significantly improve developer code comprehension across three indicative software engineering tasks by up to 12% for 87% of participants. In addition, we demonstrate that COMCAT-generated comments are at least as accurate and readable as human-generated comments and are preferred over standard ChatGPT-generated comments for up to 92% of snippets of code. Furthermore, we develop and release a dataset containing source code snippets, human-written comments, and human-annotated comment categories. COMCAT leverages LLMs to offer a significant improvement in code comprehension across a variety of human software engineering tasks.

I. INTRODUCTION

Software maintenance accounts for a significant portion of the total lifetime costs of software, with estimates ranging from 66% to 90% [1]. Within the realm of maintenance, approximately half of the costs are attributed to code comprehension [2]–[4]. Furthermore, a substantial portion of the time spent on comprehension is dedicated to reading plain code, comprising over 40% of the overall comprehension time [5]. Given these statistics, it is evident that software readability and comprehension play pivotal roles in the cost-effectiveness and efficiency of software development and maintenance processes. To address this critical aspect of the software life cycle, we present a method to improve automated software documentation by augmenting Large Language Models (LLMs) with an awareness of (1) categories and (2) appropriate locations of source code comments that are identified as helpful according to human software developers.

The importance of comments in code comprehension has been a topic of debate among researchers [6, 7]. While Börstler and Paech argue that comments primarily enhance perceived readability rather than objectively improve comprehension [7], other studies indicate that commented code is easier to understand [8]–[10] and comments are particularly useful for debugging, comprehending, and refactoring [6, 8]. Moreover, inadequate comments can substantially impede software maintainability [11]. These contrasting viewpoints imply that, while certain comments may not contribute to improved comprehension, the appropriate placement of complete, concise, and accurate comments can greatly enhance the efficiency of code reading. Consequently, the identification, placement, and generation of comments that can improve comprehension emerges as a crucial problem to address.

Automated code documentation generation, or code summarization, has seen several advancements in recent years with the goal of supplanting or supplementing manual documentation [12]–[37]. Researchers have improved performance in this task using context-aware techniques, and text classification in particular has been shown to improve performance for multi-intent comment generation [38, 39]. While previous research focuses on generating comments for function-level summaries alone, our work recognizes the critical need for inline comments within functions in a variety of contexts. Such comments play a pivotal role in clarifying complex code logic on a granular level, which has been largely overlooked as a means for enhancing comprehensibility. In this paper, we address two key challenges: (1) identifying the types and locations of comments that are most useful for developers; and (2) demonstrating improved comprehension by human developers without relying on traditional evaluation metrics like BLEU, which do not correlate well with developer performance [8] and that may not be appropriate for recent developments in LLMs [40, 41]. By leveraging human studies to guide context-aware comment generation, we enhance LLMs to meet these challenges for more granular and comprehensible documentation of source code.

In this paper, we present COMCAT, an approach to automatically generate source code comments for several levels of abstraction, illustrated in Figure 1. Given an input source code file, (1) the Code Parser splits the file into Snippets that capture commonly-used structures, like loops and variable declarations, (2) the Code Classifier predicts the most helpful type of comment to place within each Snippet, and (3) the Prompter uses an LLM to generate a comment for each Snippet and type of comment required. COMCAT outputs a well-documented source code file with each Snippet and

Skyler Grandel, Yu Huang, and Kevin Leach are with the Department of Computer Science, Vanderbilt University, Nashville, Tennessee, USA. Email: {skyler.h.grandel, yu.huang, kevin.leach}@vanderbilt.edu.

Scott Thomas Andersen is with the Department of Computer Science, Universidad Nacional Autónoma de México, Mexico City, Mexico. Email: stasen@comunidad.unam.mx

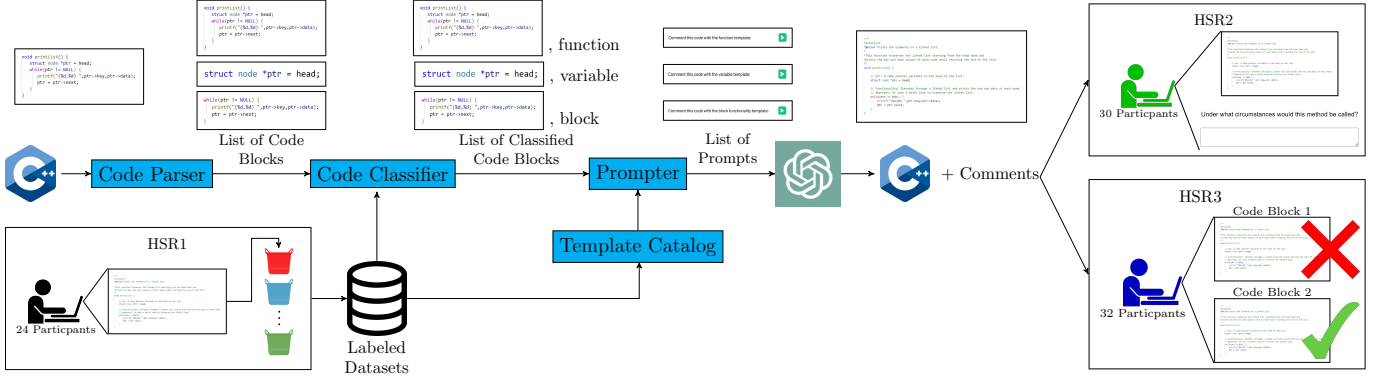


Fig. 1. COMCAT pipeline and study procedure. We use three instances of human subjects research to inform COMCAT’s design (1) and evaluate developer performance (2) and preference (3) with our tool. COMCAT takes C/C++ code as input, using a Code Parser to identify code Snippets to be commented. These Snippets are classified, and the class of each Snippet is used in combination with our Template Catalog to create a prompt for each Snippet. These prompt ChatGPT, which outputs the commented code. This pipeline is informed by developer expertise, but it is fully automated and requires no human intervention.

corresponding comment. We conduct three Human Subject Research (HSR) studies in this paper¹. First, we surveyed 24 developers (HSR1 in Figure 1) to determine which types of comments are most helpful for comprehension, from which we trained the Code Classifier and built a Template Catalog for prompting LLMs for comments. Next, we recruited 30 additional developers (HSR2 in Figure 1) to evaluate how much COMCAT improved developer comprehension in three different software engineering tasks. Finally, to determine the effectiveness of our pipeline, we recruited 32 different developers (HSR3 in Figure 1) to determine whether developers preferred COMCAT over human-written and ChatGPT-generated comments. We show that our approach significantly improves developer comprehension and that developers significantly prefer COMCAT over other approaches to generating source code comments and summaries. Finally, we release our dataset of source code, snippets, human-written comments, and anonymized human subject data for future research.

The main contributions² of this paper are:

- A set of developer-informed comment categories for comment analysis with a corresponding classifier and dataset of comments and their associated C/C++ code annotated via a HSR study with 24 participants.
- A catalog of prompts for LLMs to produce comments of each relevant comment class.
- A tool for dynamically selecting and automatically commenting code Snippets in a C/C++ file using expertise-guided classification and prompting with an LLM.
- A second human study of 30 participants demonstrating that our context-generation improves developer performance on code comprehension tasks by an average of 12% for 87% of developers.
- A third HSR study of 32 participants demonstrating that our context-generation leads to documentation that is at least as accurate and readable as what was generated by humans in 80 to 100% of cases and is preferred to standard ChatGPT in 60 to 92% of cases.

II. BACKGROUND AND RELATED WORK

Source code documentation is an active area of research that we build upon by improving analysis and generation of software comments. In this section, we consider two lines of related work: (1) analysis and categorization of software documentation, and (2) techniques for automatically documenting source code. We acknowledge the contributions of these works and leverage their insights to better generate comments.

A. Code Documentation Quality Analysis and Categorization

Other researchers have conducted a number of studies on comment quality evaluation. The most similar to our work is a study by Steidl et al. [42], which produces a quality assessment model and a set of systematic metrics for evaluation. They further classify comments, largely by the code they annotate, into seven categories: copyright comments, header comments, member comments, inline comments, section comments, code comments, and task comments. This work led to a machine learning model that automatically classifies comments into these categories. A similar work by Chen et al. [38] uses manual categorization by purpose into six categories to improve code summarization in a similar way to our work, though they only consider comments that describe entire methods. In this paper, we re-examine automatic comment classification as a tool for improving comment generation and assessing comment quality using a more detailed categorization schema that better distinguishes inline comment types.

Khamis et al. [43] present JavadocMiner, a tool for automatically calculating the language quality and cohesion with code of Javadoc comments. It additionally analyzes an entire project to calculate how well commented it is. This study produces a number of metrics and heuristics to determine how well a function comment documents a method and how well a file or project is documented overall. To the best of our knowledge, we are unaware of any previous attempted use of HSR for comment classification or any attempts at experimentally evaluating the quality of comments. We address this gap by investigating the quality of comments generated by LLMs and how it can be improved by Transformer-based classification.

¹All HSR in this paper was exempted by our institution’s IRB.

²Our replication package can be found at: https://osf.io/tf2eu/?view_only=4dcf8efa50a346dc96d28b139b0d3b90.

B. Automated Code Documentation

Automatic documentation typically involves three approaches: (1) template-based, (2) information retrieval, and (3) learning-based. We discuss each below.

Template-based tools use a predefined layout with rules to produce documentation. Some have used natural language templates to capture details of functions [33] while others have used heuristics [28, 31], contextually informed templates [27], and stereotyping with a code analysis framework [12]. Buse and Weimer further used templates to document code changes using symbolic execution [44] and exceptions [45].

Information Retrieval tools seek out existing documentation to copy into new code. Haiduc et al. used latent semantic indexing and vector space modeling to generate comments for functions and classes [20, 21]. Eddy et al. contributed a hierarchical model [17] while Rodeghero et al. improved upon it by using eye-tracking data to inform statement and keyword selection [32]. Code clone detection has also been leveraged to mine StackOverflow for comments by Wong et al. [36, 37].

Learning-based tools typically use deep learning techniques to learn latent features from source code by training on existing code/summary pairs. CODE-NN, an LSTM-based neural network by Iyer et al., was trained on StackOverflow code to summarize C# and SQL code [24]. Convolutional Neural Networks [14] and Neural Machine Translation [15] have also been used for code summarization. Hybrid approaches combining code with ASTs [34] or lexical and syntactical information [22, 23] have also been popular.

Other works have used Transformer-based methods. CodeBERT [18] and CodeT5 [35] are pre-trained transformer models based on BERT [16] and T5 [46] respectively. GPT-2, CodeGPT-2, and CodeGPT-adapted [13, 47] have all also been evaluated on this task using the decoder-only GPT-2 base model. The Code Structure Guided Transformer [19] incorporates code structural properties into the transformer to improve summarization performance. CoTexT [30] learns the representative context between natural and programming languages and performs several downstream tasks, including code summarization. PLBART [13] is a sequence-to-sequence model pre-trained on a large set of Java and Python functions and their textual descriptions. Liu et al. [26] uses a retrieval-augmented GNN to generate comments for C/C++ source code. Hybrid approaches have also been used, such as incorporating relevant codes/summaries retrieved from a database [29]. Others have used LLMs like Codex, a GPT-3 based tool evaluated on summarization by Kahn and Uddin [25]. Similar to our work is a study by Geng et al. [39], who combine Codex with a taxonomy of comments from Chen et al. [38] to produce multiple summaries from different perspectives. We build on this by extending the taxonomy to enable inline comment generation, introducing dynamic code Snippet selection, and evaluating human perceptions of our resulting comments. CodeBERT, CodeT5, GPT-2, CodeGPT-2, CodeGPT-adapted, CoTexT, and Codex were all trained and/or evaluated using CodeSearchNet [48], a dataset of functions and their associated documentation in six languages.

Our approach dynamically selects code Snippets and cor-

responding comment types to generate the most helpful comment as informed by human developers. Our approach can apply both to inline Snippets of code as well as entire functions [33, 43]. To the best of our knowledge, our work is the first to generate both types of comments enabled by comment type classification.

III. COMMENT CLASSIFICATION APPROACH

A critical foundation of our approach is to identify the best type of comment to place in source code as well as where to place each comment. In this section, we present our approach for classifying comments, which we use in our overall pipeline to provide targeted, comprehensible comments in source code. This work enables developers to thoroughly and accurately document code with software comments automatically, saving substantial monetary and temporal costs associated with software development and maintenance [1]–[5]. With this goal in mind, the following subsections detail our HSR-based classification and comment templates. These components form the basis for our expert-guided context generation to be used in our COMCAT pipeline for fine-grained software documentation.

The code and comments in this study (and in all HSR studies in this paper) were scraped from C/C++ GitHub projects originating from the DARPA MUSE project [49]. This dataset was formatted with Clang to ensure a consistent style [50].

A. Classification Schema

To establish appropriate comment types for this work, we undertake a systematic process of comment categorization based on the information comments provide about their associated code. We build on previous works by identifying distinct comment types that capture the semantics and purpose of each comment, with the end goal of providing more descriptive inline comment types than previous works to encapsulate the varied information and structures of such comments [38, 42]. To ensure the accuracy and generalizability of this process, two researchers independently categorize comments, starting with zero categories and creating a new category when no existing category applies to the current comment. Following accepted qualitative methods [51, 52], the categorization process continues until saturation³ is reached, ensuring comprehensive coverage of comment types.

Following this process, we take the union of the individual categorizations of the two researchers, and equivalent categories are consolidated to form a comprehensive schema through discussions and iterative refinement, resulting in a schema of 12 comment types. To further validate this schema, we conducted HSR1 (described in Section III-B). HSR1 revealed four of the initial 12 types are perceived to be useful for improving developer understanding of source code, finalizing our schema with the following classes:

- **Function** comments describe an entire function. They tend to note parameters and return values.

³In this study, *saturation* refers to the point where no new categories have been added to the codebook after at least 100 consecutive comments have been categorized without creating a new type.

- **Variable** comments describe a variable, constant, or literal. They often note what a variable represents.
- **Snippet Functionality** comments are inline comments that summarize or describe the functionality of code.
- **Branch** comments describe possible branches of execution. This also includes preconditions for branches.

B. HSR1: A Human Study of Comment Classification

To categorize our dataset of comments, we conduct HSR1, a survey of 24 student and professional developers. We recruit participants from both student and professional populations of developers with at least 2 years of C/C++ experience from various universities and companies through email solicitations and online crowdsourcing⁴. We employ a C/C++ prescreening test to ensure adequate programming aptitude. This prescreening consists of questions found by Sillito et al. to be relevant to developer comprehension, similar to the questions employed by our study in HSR2 [53]. The survey is administered online via a URL given to participants. Participants are provided with a training video giving standard and edge-case examples of comments and their associated categories, along with an explanation of each categorization.

In HSR1, we ask participants to classify comments drawn randomly from our dataset according to the categories from our schema. Participants also have an option to note that a given comment does not fit any category. We further ask participants to indicate whether the comment aids in their understanding of the code. Participants are given the entirety of a file for context and indicate a ranked choice of up to three categories for a given comment to resolve disagreements and to allow for comments that exhibit qualities of multiple types. We also ensure that at least two participants classified every example in the dataset. Although participants could report new comment types if they chose, we found that such cases consisted of existing types.

We further calculate the inter-rater reliability of the results using Krippendorff’s Alpha (α) [54]. This statistic is a generalization of Fleiss’ Kappa to inconsistent numbers of respondents [55]. This results in an α of 0.56 across 24 participants and 12 nominal categories. This can be interpreted as a “moderate agreement” for a task where chance agreement is unlikely due to the large number of categories. While this indicates reliable data, examples can exhibit characteristics of multiple classes, resulting in occasional disagreement. We resolve such disagreements by selecting the ranked choice majority category to be the accepted class and resolving any remaining disagreements through discussion among two authors. HSR1 resulted in a dataset of 952 examples of comments and associated human-annotated comment categories. These comment categories were ultimately used for training a comment classifier used in subsequent parts of our pipeline.

Having established a schema of comment categories, we next define a set of comment templates. Recall that our eventual goal is to categorize the type and location in which

to place comments in source code. Each comment type is associated with a template prompt used to prompt an LLM to generate a comment of that type in a specific context. We use developer expertise to guide the comment type that we use to describe a given code Snippet. However, we found that not every type of comment is useful for the specific purpose of comprehending code. Therefore, we constrain our comment types to include only those that developers in HSR1 found useful for comprehension in a majority of cases. Thus, we create templates based on the structure of comment type that was considered useful for comprehension in HSR1, and discuss each template below.

C. Template-Based Prompting

Recall that our goal is to automatically document source code based on the best predicted comment type and location. Having already established a developer-validated schema of useful comment types, we next design prompt templates for LLMs, which we treat as black box state-of-the-art comment generators [25]. By augmenting the prompt with a predicted location and helpful comment type, we can target the specific type of comments for the LLM to generate that are more likely to be helpful for code comprehension according to HSR1.

In Section III-B, we developed a scheme of 12 comment types, and asked participants to annotate which comment types they believed were most helpful for comprehension. Thus, for building our prompt templates, we consider only the comment categories that were judged to be useful by a majority of participants in HSR1. We further acknowledge that practitioners prefer documentation that explains what the associated code is doing as well as why [56]. Therefore, we consolidate “Reasoning” into our 4 templates by prompting for the code’s purpose so that all of our documentation explains both the “what” and the “why” for some given code. We further note that some developers consider “Use Guideline” comments to be useful [39], so we include use-guideline elements in our “Function” template.

To ensure that our LLM (i.e., ChatGPT-3.5) produces the comments in the format we desire, we use the Template prompt pattern [57]. To develop our templates, we use common structures of each comment type in our dataset and the Flipped Interaction pattern [57] in combination with testing ChatGPT for its adherence to the prompt, output length, and comment cohesion with the input code. These trials resulted in templates for function, variable, Snippet functionality, and branch comments, which can be found in Table I. All templates incorporate the reasoning type to explain implementation decisions for all input code by prompting for the code’s purpose.

The templates in Table I are included in a prompt that instructs ChatGPT to generate a comment for the given code Snippet using the template. We further use the OpenAI API system prompt to indicate ChatGPT’s role as a comment generator, provide the entirety of the file for context, and limit the length of the output comments. We ultimately use these templates and their associated prompts to produce a targeted comment with content and structure that fits the semantics of a particular code Snippet.

⁴We specifically gather 10 participants in HSR1 from Mechanical Turk and 15 participants in HSR2 from Prolific for crowdsourcing. All other participants were student volunteers from various universities.

TABLE I
OUR TEMPLATE CATALOG. THIS CATALOG ASSOCIATES OUR COMMENT TYPES WITH A TEMPLATE FOR PRODUCING A COMMENT OF THAT TYPE. TEXT IN BRACKETS SHOULD BE REPLACED WITH INFORMATION SPECIFIC TO THE CODE BEING COMMENTED.

Template Catalog	
Function	<pre>/** * [Function name] * @brief [Brief description of the function] * * [Provide a detailed explanation of the function's purpose and functionality] * * @param [param1] [Description of the first parameter] * @param [param2] [Description of the second parameter] * @return [Description of the return value] * * @exception [ExceptionType] [Description of the exception and when it can occur] */</pre>
Variable	<pre>// [Variable/Constant/Literal Name]: [Description and purpose].</pre>
Snippet Functionality	<pre>// Function: [Functionality and purpose]. // Approach: [Approach or strategy].</pre>
Branch	<pre>// Cond: [Conditions] // Function: [Functionality and Purpose]</pre>

IV. COMCAT PIPELINE APPROACH

In this section, we present the COMCAT pipeline architecture and detail its components. A visualization of this pipeline is given in Figure 1. Taken as a whole, COMCAT accepts a C/C++ file and outputs the same file with comments describing the functionality, purpose, and reasoning of the code embedded throughout. COMCAT fully automates the process of commenting C/C++ files to save costs associated with documenting code [1]–[5] and conferring the benefits of well-documented code, like eased comprehension, debugging, refactoring, and reuse [6, 8]. Quality is crucial to a comment’s ability to confer these benefits and COMCAT’s purpose is to ensure consistent, accurate, high-quality comments [8, 11, 42]. We accomplish this through careful preprocessing of input files to identify appropriate locations in which to generate developer-preferred comments, as well as the careful design of LLM prompts based on developer guidance. While COMCAT automatically comments files without the need for any human intervention, users may validate and edit the results to mitigate concerns about AI reliability. In contrast to prior art, this approach strikes a balance between sufficient documentation at a granular level (as opposed to function-level summarization seen in prior work) and succinct documentation that is objectively helpful for developer comprehension (as opposed to AI approaches that are only internally validated with metrics like BLEU and ROUGE).

To achieve our goal, we (1) split an input source file into a number of code Snippets. Next, we (2) determine an appropriate comment type to document each Snippet. Then, we (3) select the most appropriate prompt template based on the selected comment type. Next, we (4) fill the prompt template with contextual information from the code Snippet and type. Finally, we (5) feed each prompt to ChatGPT to build the final commented file. Therefore, this pipeline consists of five components: the Code Parser, Code Classifier, Template Catalog, Template-Based Prompter, and ChatGPT. We discuss each below.

A. Code Parser

The Code Parser splits code into parts that ChatGPT will comment so that Snippets within the file can be documented. It takes C/C++ code as input and outputs a list of code Snippets to comment. Not every line from the source code will be included in the output, and code Snippets may overlap. As shown in Figure 1, code Snippets could be selected from various levels of abstraction, such as whole functions, single statements, or loop structures.

The Code Parser is implemented using the Clang C language family frontend for LLVM [50]. This allows us to parse input code and identify constructs defined by several lines of code or several scopes within the code via an LLVM pass. This Code Parser selects every instance of the following C/C++ constructs defined by LLVM to be commented: functions/methods, conditionals, loops, try-catch-finally blocks, and declarations (e.g., of classes, structs, variables, etc.). We group associated Snippets together, so an if-else statement is considered a single Snippet. This functionality is provided entirely by LLVM. This component outputs a list of such Snippets to be passed on to the Code Classifier so that each code Snippet can be matched with a comment type (cf. Section III-B) to describe it.

B. Code Classifier

The Code Classifier is a Transformer model that is trained to determine which of the four kinds of comment would be appropriate to describe the code Snippet. These four comment classes are (1) function comments, (2) variable comments, (3) Snippet functionality comments, and (4) branch comments (recall these were defined in our comment type schema in Table I). The Code Classifier takes a list of code Snippets from the Code Parser and outputs a list of classified Snippets. Each Snippet is passed to the prompter so that it can select the appropriate prompt to feed to ChatGPT to produce a comment for that Snippet.

In this subsection, we describe the details of the architecture and training for the Code Classifier model. This is a multi-

layer bidirectional Transformer model [58] trained specifically for the task of classification with a neural network classifier head. We use the 125M parameter model architecture of Microsoft’s CodeBERT [18], which itself is based on BERT [16] and RoBERTa [59]. We elide details of the Transformer architecture and the associated training methodology, which can be found in the relevant works [16, 18, 59]. The Code Classifier additionally has a sequence classification/regression head that uses the same architecture and training methodology as the RoBERTa model for sequence classification [59].

The Code Classifier is trained using 90% of the dataset gathered in Section III-B and is evaluated on accuracy and f1-score using the randomly-selected, held-out 10% of the dataset. We pretrained on the Masked Language Modeling (MLM) task to improve syntactic understanding of the domain. This is the same method used to train CodeBERT on the MLM task [18]. This improves the feature vector intermediate representation generated by the base Transformer. This vector is used by the classification head to represent the syntax of the input sequence in vector form, which can be classified using more classic deep learning techniques [60]. We perform this step to improve the model’s understanding of C/C++ and of smaller code Snippets within functions. This is necessary because CodeBERT is trained on only entire functions and their associated comments in a variety of languages not including C/C++. While CodeBERT generalizes to out-of-scope languages [18], the model could be misled by certain language idiosyncrasies like whitespace in Python or naming conventions in Java. This step results in a 24% increase in accuracy and f1-score.

Next, we train the Code Classifier model on our classified dataset, using code (with comments removed) to determine the class. The Code Classifier only classifies code into one of the first four types, because these are the only types that assist with comprehension as found by HSR1 and as described in Section III-C. The Code Classifier does not include many of our comment types because its purpose is to recommend a comment type to describe a given code Snippet, and we do not want to recommend adding, for example, a TODO comment, as such a comment would not aid comprehension.

We calculated accuracy and f1-scores resulting from this training by using 10% of the dataset held-out as testing data and taking the median of 5 trials, as with the evaluation of RoBERTa for the classification task [59]. We find that the Code Classifier model achieves an accuracy and f1-score 0.96 for this task. This shows that our models can classify comments effectively in spite of our observation that human-written comments often exhibit the qualities of multiple categories simultaneously. This enables the Code Classifier to effectively determine which comment template is to be used to describe a given code Snippet within the COMCAT pipeline, enabling dynamic code Snippet selection for more fine-grained comment generation and improved readability.

C. Template Catalog

The Template Catalog in this pipeline is a fixed mapping of comment types to comment templates. These templates detail

the structure of their associated comment type for ChatGPT using the Template prompt pattern [57] and they are designed to answer questions programmers ask during software evaluation tasks [53, 56] (Section III-C). The Template Catalog consists of four comment templates corresponding to each of the four classes that could be output by the Code Classifier. The purpose of this component is to maintain a set of templates that can be incorporated into prompts to outline the structure of the comment that we want to generate for a specific Snippet of code. This component is queried by the prompter using the class of the current code Snippet as the key and giving the associated template as the associated value.

D. Template-Based Prompter

The Template-Based Prompter constructs a list of prompts that each requests comments of a particular format to describe the code. It takes a list of code Snippets and corresponding comment type classifications as input and outputs a list of prompts. The prompter takes each classified code Snippet, queries the Template Catalog to retrieve the comment template associated with code Snippet’s class, and crafts a prompt consisting of ChatGPT’s instructions, the template, and the code Snippet. These instructions detail ChatGPT’s task to generate a comment, specify output that describes the code’s functionality and purpose, and explain how to use the template. We further limit inline comment length in these prompts to 30 words following previous work suggesting that comments over 30 words should be reserved for descriptions of entire functions or files [42]. The prompter is important because careful prompt engineering is critical to contextualizing requests and ensuring specific qualities of LLM outputs [57].

E. ChatGPT

To produce comments describing code snippets from an input file, we query ChatGPT with prompts output by the prompter. ChatGPT is a state-of-the-art LLM capable of many tasks, including code summarization [61]. It enables us to specify a complex output while providing context for the input code to produce an accurate and comprehensible comment.

We interact with ChatGPT for a single conversation for the process of commenting a file. We first use a system prompt to guide the model’s behavior throughout the conversation. Our system prompt specifies ChatGPT’s role as a software comment generator and provides the entire file to be commented. We give ChatGPT the entire file prior to the individual code Snippets so that it has context for those Snippets within the file that it will comment, as context has been shown to improve code comment generation [30, 62, 63].

Following the system prompt, we use user prompts to generate comments for individual code Snippets. User prompts are intended for standard conversational interactions with ChatGPT, and we use them to make requests with prompts generated by the Prompter. These prompts are fed to ChatGPT in sequence and a comment is generated for each prompt. This generates every comment to be included in the newly-commented file. Comments are then placed on the line above the associated code Snippet in the original file. Finally, the complete file with embedded comments is output to the user.

V. EVALUATION AND RESULTS

In this section, we evaluate COMCAT’s effectiveness at producing accurate and readable comments. We conducted two additional HSR studies to evaluate our pipeline according to (1) developer performance and (2) developer preference. We evaluate several research questions based on our findings:

- RQ1** Does COMCAT lead to improved developer performance on code comprehension tasks?
- RQ2** Which code comprehension tasks are most aided by COMCAT?
- RQ3** Do developers perceive COMCAT comments as higher quality than human-written and standard ChatGPT-generated comments?

We present the results for each of these research questions and discuss their interpretations. All participants in these studies were recruited from student and professional populations of developers with at least 2 years of C/C++ experience from a variety of universities and companies through email solicitations and online crowdsourcing. All participants must pass a C/C++ code comprehension prescreening survey. This survey is administered online via a URL given to participants.

A. RQ1: Developer Performance

For this research question, we seek to understand if COMCAT leads to objectively improved developer performance on code comprehension tasks. We evaluate developer performance on such tasks given code that has been commented by humans and COMCAT. To that end, we conduct a study of 30 developers (15 students and 15 professionals) to complete a set of indicative code comprehension and debugging tasks. This is a 16 question survey; each question consists of a syntax-highlighted code window and a question drawn from three types: short answer, code writing, or debugging. We selected code for each question from our dataset (Section III-B) that were between 15 and 50 lines long and contained at least one of the four comment types from our schema. We further validate that each code snippet does not require external information (e.g., a reference to an external file) about its project to answer the questions we ask. Questions cover a variety of typical software engineering topics, including string manipulation, data structures, memory management, and concurrency. For each question, each participant is randomly assigned either the COMCAT-commented or human-commented variant of the code. The code is identical between cohorts; only the comment changes. The human-written comments are those written in the original GitHub repositories from which we drew the code in our dataset — we consider these to be ground truth, high quality human-written comments. With this design, we seek to evaluate whether the comments produced by our approach lead developers to be more productive or better comprehend source code than with baseline human-written comments of the same code. Below, we discuss each type of task given to participants in this study.

Short Answer questions are open-ended questions intended to assess a participant’s ability to comprehend the code. Each participant is given 10 code files, one at a time, and is asked to answer three short answer questions per file.

These questions are taken from those found to be relevant to developer comprehension by Sillito et al. [53] and are adapted to the specific file shown to the participant. This follows methodology from previous work [8]. Specifically, we adapted the following questions from Sillito et al.:

- Q1. Under what circumstances is this function called?
- Q2. How can we tell that this function has executed correctly?
- Q3. What will be the direct impact of this change?

These questions were chosen because they relate to the comprehension of single functions; we did not consider questions that relate to entire classes or software ecosystems.

Code Writing questions involve writing a simple function in the same namespace as the given code that accomplishes some given task related to that code. The participant gives their answer in a syntax highlighted editor. In each of the three questions of this type, the task required either a simple application of the given code that could be accomplished in 5 LOC or less, or it required reusing logic from the given code. Here, we test a participant’s practical understanding of the code resulting from the comments, independent of their ability to implement complex logic.

Debugging questions require participants to debug a given code sample. They are given a textual description of the bug that explains the problem and how it has affected the output of the function. The code is then presented in an syntax-highlighted editor for the participant to modify. The bugs were injected into this code by making a single line edit, ensuring a simple correct answer. Here we measure comprehension of the code as a result of the comments, independent of a participant’s ability to implement complex logic.

Since participants could provide open-ended answers, we developed a rubric to grade each snippet-question pair to provide a quantitative assessment of participant data. For each question, we mark an answer as correct or incorrect based on the rubric. This rubric gives latitude for minor mistakes unrelated to comprehension of the snippet in the case of code writing and debugging questions. Three raters graded all responses in this study and we achieved high inter-rater agreement [55] for our binary correctness ratings ($\kappa=0.81$, “high agreement”). Disagreements in correctness were resolved through discussions and majority results.

We divide our data into two groups: responses given COMCAT-generated comments, and responses given human-written comments. We consider rater-assessed correctness for each response and compare the two populations of responses. We note a significant difference in correctness between these two populations ($p<0.001$), finding that correctness improves by 12% given COMCAT-generated comments over the human-written alternative. We further find that 83.33% of participants perform better given COMCAT-generated comments and an additional 6.33% perform equally well between the two versions. Participants given human-written comments scored an average of 71.90% correct, while those given COMCAT-generated comments scored an average of 83.88% correct.

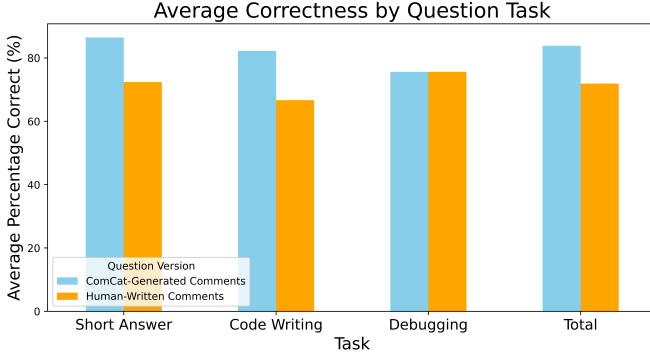


Fig. 2. Results from HSR2. This shows the difference in correctness between COMCAT-generated and human-written comments for each question type.

Therefore, we find that both professional and student developers find code easier to comprehend when COMCAT documents the code compared to developer-written documentation. Thus, our expertise-guided context can improve automated comment generation to aid developers in comprehension and maintenance.

B. RQ2: COMCAT Performance on Varied Tasks

Following our evaluation of COMCAT’s effect on developer performance on code comprehension tasks, we further investigate performance on individual comprehension tasks. Here we evaluate participants in our human performance study (cf. Section V-A) on our three question types (i.e. short answer, code writing, and debugging). We again measure the change in correctness between responses resulting from COMCAT-generated versus human-written comments, but we isolate each task to determine how performance on that task is affected by our intervention. The results from this investigation, as well as the overall change, are summarized in Figure 2.

1) *Short Answer Task*: This task involved answering open-ended questions found by Sillito et al. [53] to be important questions for comprehension that are commonly asked of developers by project leads. We find that participants answer these questions correctly significantly more often given COMCAT-generated comments ($p < 0.001$). More precisely, the correctness is improved by 14.08% from 72.38% correct (for human-written) to 86.46% correct (for COMCAT). Furthermore, 90% of participants performed better on this tasks using COMCAT as well. This indicates that COMCAT outperforms human-written comments in helping developers answer project-relevant questions. We attribute this to inline comments assisting developers in answering more focused comprehension questions.

2) *Code Writing Task*: This task involved solving a short coding problem by using given functions based on documentation for those functions. We find that participants answer questions correctly more often given COMCAT-generated comments; however, the difference is not significant ($p = 0.097$). Here, correctness is improved by 15.55% from 66.67% correct (for human-written) to 82.22% correct (for COMCAT). While these results do not reflect significant improvements in code writing, these results do at least suggest that COMCAT generates comments that are no worse than ground-truth comments

written by developers. We attribute this to our function-type comments aiding developers in function interface use by describing parameters and return values, and our inline comment types further assist developers in reusing the logical intuition from Snippets within the code.

3) *Debugging Task*: This task requires participants to debug a code snippet given a natural language description of the buggy behavior. This requires participants to identify and fix a single buggy line of code. We find that, on average, participants performed exactly the same given both versions of the commented code, so we cannot make any conclusions regarding the superiority of either version in aiding participants for this task. Specifically, participants averaged 75.6% correctness when given both the COMCAT-generated comments and the human-written comments. Thus, we can only surmise that COMCAT-generated comments are likely no worse than human-written comments. We attribute this to COMCAT-generated comments describing what the code is actually doing, not what it is intended to do. This could explain why we do not see improved correctness for this task.

Overall, our investigation into COMCAT’s performance across varied comprehension tasks reveals nuanced insights into its effectiveness. The short answer task showed significant results in COMCAT’s performance versus that of human-written comments, indicating improved comprehension. Meanwhile ComCat provided equally helpful documentation for code writing and debugging tasks.

COMCAT can improve developer correctness on project-relevant questions without detrimentally impacting debugging or code reuse. This is an improvement over previous function-level summarization, as such comments are less effective than human-written comments [8].

C. RQ3: Developer Preference

For this research question, we evaluate developer perceptions of COMCAT’s comments. We assess developer preference for COMCAT versus human-written comments and standard ChatGPT through two main evaluation tasks: describing individual code Snippets and automatically commenting files. We designed another human study of 32 developers focusing on their comment preferences. We do not compare COMCAT’s performance with models specifically trained for code summarization in C/C++ because such models are only evaluated on summaries of entire functions and are incapable of parsing files for code Snippets to summarize (e.g., specific declarations, statements, or loop constructs). Moreover, these models are trained to replicate ground truth human-generated comments, which we include in this study anyway. We choose ChatGPT for our LLM comparison rather than other LLMs, like Google’s Bard, because its architecture is the state-of-the-art in code summarization [25] and is the most widely used for AI programming assistance through GitHub Copilot (based on OpenAI’s Codex) [25, 64]. Therefore, we compare to human-generated comments and ChatGPT, which we observe might produce overly verbose documentation that might not aid in comprehension, as we show with this study.

Commonly, software that is documented by humans is sparsely-commented and existing research lists comment com-

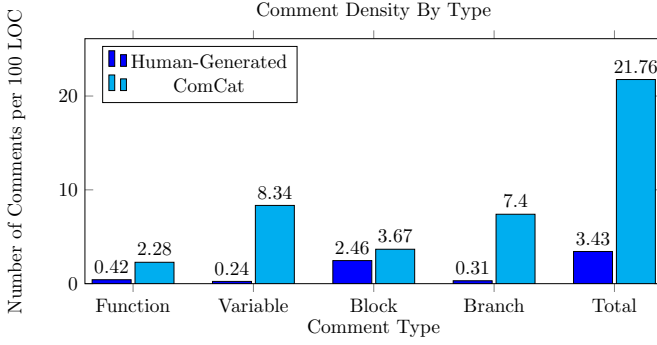


Fig. 3. Density of comment types in our dataset. Here, density is the number of comments per 100 LOC. COMCAT improves comment density over humans, addressing the problem of sparsely commented, minimally maintainable code without cluttering code with excessive documentation.

pleteness and missing or insufficient comments as major issues with existing documentation [6, 42, 65, 66]. The goal of this pipeline is to automatically improve code readability and maintainability by generating comments of various types. This requires a balance in comment density to ensure the documentation is complete, but is not so verbose as to clutter the code. We illustrate this problem in Figure 3, showing that COMCAT generates a greater density of comments for more complete documentation than what is produced by human developers⁵. Figure 3 further shows that we generate a range of comment types more completely than humans, and humans tend to produce very few comments that explain variables or branches. COMCAT further documents every instance of a function or method with a function comment, where humans in this data documented only 18.42% of functions. We similarly calculated these metrics for comments produced by standard ChatGPT. We found that while it documents nearly every function with a function comment, it rarely produces variable and branch comments, potentially neglecting important semantics for comprehension. We show in our experiments that COMCAT’s comments are subjectively individually as good as those generated by humans and do not clutter code with excessive documentation, as evidenced by developer preference for comments, striking the balance between completeness and succinctness that we desire. The following subsections describe this evaluation, indicating that COMCAT can improve documentation through increased completeness without sacrificing individual comment quality.

1) *Experimental Design*: For the first preference evaluation task in this study, we assess COMCAT’s ability to generate comments that accurately and comprehensively describe relevant information about individual code Snippets within source code according to developer preferences. This shows that COMCAT’s comments are individually at least as preferred as those generated by humans and standard ChatGPT. We randomly select code Snippets and their associated human-generated comments from the held-out test data (Section IV-B) in our dataset (Section III-B). However, we ensure that the human-written comments from the dataset are one of the four types of comments we established in our schema. Because the code Snippets are already chosen, this task does not evaluate

the Code Parser; instead, single code Snippets are passed on to the next stage of the pipeline. Testing against human-written comments allows us to explore the pipeline’s usefulness in supplementing or supplanting manual commenting. Here we demonstrate COMCAT’s ability to save developer effort by automatically commenting code Snippets in a way that is at least as accurate and readable as human-generated comments. In separate trials for the same task, using the same code Snippets, we compare to comments produced by standard ChatGPT to evaluate our pipeline against a state-of-the-art LLM, demonstrating the effectiveness of our model augmentation and capturing improvements to ChatGPT’s comment generation. For this comparison, we prompt ChatGPT as similarly as possible to COMCAT without the assistance of our pipeline to mitigate prompt selection biases.

For our second preference evaluation task, we assess our pipeline’s ability to comment entire C/C++ files and to see whether developers prefer these comments over other automated comment generation techniques and human-written comments. We use a diverse set of source files, covering various software domains and complexity levels, drawn randomly from our dataset. The C/C++ files are fed into our pipeline, and the output is compared to comments written by the code author and generated by standard ChatGPT. We select code Snippets and human-generated comments in the same way as in the evaluation task for individual comments and we again prompt standard ChatGPT as similarly as possible to COMCAT.

HSR3 is a survey of 32 developers, who provide subjective feedback on the quality and relevance of the generated comments compared to human-generated and Standard ChatGPT-generated comments. Every participant in this study answered every question for the trial they were given. For each question in this study, participants are presented two windows containing C/C++ code and associated comment(s), where the code is identical in both windows, but the comment(s) are generated differently. Participants are not informed of the method used to generate each comment. We ask participants to indicate a preference (or no preference) between the two alternatives for which example they find to be more “readable, accurate, and useful for understanding, editing, debugging, or reusing the associated code.” At the end of each trial, participants are asked to explain their preferences and describe qualities they tended to like or dislike in each option. Participants choose between COMCAT’s comment and either one produced by Standard ChatGPT or a human-written comment from our dataset. This design allows us to garner specific, comparative insights into the utility and quality of the ComCat-generated comments according to developer preference, aiming to ascertain developers’ subjective preferences about our pipeline in enhancing code comprehensibility.

2) *Preference Study Results*: Here, we discuss the results of HSR3, which shows how COMCAT aligns with developers’ expectations. A summary of our findings can be found in Tables II and III. We measure the proportion of questions for which a majority of participants indicated preference for COMCAT in each of our trials (c.f. Table II), along with the mean percent of questions that each participant answered with preference for COMCAT (c.f. Table III). Higher or

⁵This data was gathered from our dataset, described in Section III. Every file contained at least one human-written comment.

TABLE II
RESULTS FROM OUR DEVELOPER PREFERENCE SURVEY. THIS SHOWS THE PERCENT OF QUESTIONS FOR WHICH A MAJORITY OF PARTICIPANTS INDICATED PREFERENCE FOR COMCAT OVER COMMENTS GENERATED BY HUMANS AND STANDARD CHATGPT.

Developer Preference for COMCAT		
Task	vs. Humans	vs. ChatGPT
Individual Comments	80%	92%
Entire Files	100%	60%

TABLE III
FURTHER RESULTS FROM OUR SURVEY. THIS SHOWS THE AVERAGE PERCENT OF TIMES EACH PARTICIPANT PREFERRED COMCAT OVER COMMENTS GENERATED BY HUMANS AND STANDARD CHATGPT.

Developer Preference for COMCAT		
Task	vs. Humans	vs. ChatGPT
Individual Comments	66% ($p < 0.001$)	78% ($p < 0.001$)
Entire Files	82% ($p < 0.001$)	56% ($p = 0.082$)

equal preference for comments generated by our pipeline over human-generated comments may suggest COMCAT’s comments could replace or augment human commenters in such situations. Higher preference for COMCAT over standard ChatGPT indicates improvements over the state-of-the-art and showcases the subjective effectiveness of our pipeline’s context. Table III shows results calculated by participant. In this Table, we calculate p-values for each trial and find all trials to be statistically significant, except for the instance where we comment entire files and compare to standard ChatGPT, which remains inconclusive. However, we can conclude that COMCAT is no worse than standard ChatGPT in this case.

Our results indicate that the COMCAT pipeline is effective at generating informative and relevant comments according to the standards of experienced developers. We find that for non-trivial input code Snippets, COMCAT’s comments are at least as preferred as human-generated comments in 80% of cases individually, and in 100% of cases for entire files where humans tend to under-comment. COMCAT further produces comments that are preferred to those produced by standard ChatGPT in 92% of cases for individual comments and 60% of cases for entire files according to our results.

Developers find COMCAT’s comments to be more useful for code reading compared to human-written and standard ChatGPT-generated comments, showcasing the pipeline’s practical benefits in assisting with code comprehension and documentation tasks.

VI. DISCUSSION

Our results demonstrate COMCAT’s effectiveness in automating comment generation, showing that our pipeline can improve developer comprehension of source code. In this study, we found that professional and student developers find code easier to comprehend when COMCAT documents the code compared to when developers write documentation. Participants also prefer COMCAT to human-written documentation in subjective comparisons. We intuit that this is due to appropriate comment structure and density, where human-

written documentation can be sparse and is often “written only for the author’s understanding,” as participants note in HSR3.

In addition to the analyses described in Section V, we considered several other potential hypotheses that did not yield significant results. We examined expertise (measured by years of programming experience) within our participant cohorts in HSR2, and found no significant difference in the change in correctness when given COMCAT-generated versus human-written comments ($p=0.21$). Our current results suggest that COMCAT is similarly helpful for both students and professionals (i.e., novices and experts). Next, we also recorded and analyzed the time taken to complete each question for each participant in HSR2, but found no significant change in time taken ($p=0.17$). Thus, we conclude that COMCAT has no significant effect on the time taken to comprehend code when compared to code that is commented by humans.

A. Threats to Validity

While our evaluation provides key insights, there are potential threats to the validity of our approach. First, our dataset of C/C++ code and comment types used for training and evaluation might have biases inherent in the code and comments scraped from GitHub projects. We further use subjective human raters for this data to determine comment types, developer preferences, and participant correctness. We mitigate these threats by randomly choosing code Snippets from well-maintained repositories and we prescreen participants for C/C++ aptitude and experience. We further calculate inter-rater reliability metrics to ensure sound agreement.

Furthermore, while we focus on comment generation in C/C++, we expect that the insights derived from this research are applicable to other programming languages due to previous code summarization work that is applicable across languages and the generality of ChatGPT [13, 18, 25, 30, 35, 48, 67]. Our evaluation of ChatGPT is also reliant on prompt quality. To ensure a fair comparison, we use best practices for prompt engineering [57] and we prompt standard ChatGPT as similarly as possible to COMCAT’s variant.

VII. CONCLUSION

In this paper, we present the COMCAT pipeline, which uses expertise-guided context generation from human subjects research to automate the generation of inline and function-level comments. Our evaluation demonstrates the effectiveness of this context generation in enhancing code readability and comprehension, surpassing human-generated comments’ ability to assist developers in answering project-relevant questions. By following developer preferences for comment types and structures, COMCAT generates informative and relevant documentation that aids developers in understanding software. Ultimately, COMCAT represents a step towards more effective software maintenance and documentation, offering tangible benefits on the lifetime costs of software, and serving as a compelling solution for modern software engineering problems.

REFERENCES

- [1] L. Erlikh, "Leveraging legacy system dollars for e-business," *IT professional*, vol. 2, no. 3, pp. 17–23, 2000.
- [2] J. R. Foster, "Cost factors in software maintenance," Ph.D. dissertation, Durham University, 1993.
- [3] V. Nguyen, "Improved size and effort estimation models for software maintenance," in *2010 IEEE International Conference on Software Maintenance*. IEEE, 2010, pp. 1–2.
- [4] S. W. Yip and T. Lam, "A software maintenance survey," in *Proceedings of 1st Asia-Pacific Software Engineering Conference*. IEEE, 1994, pp. 70–79.
- [5] T. D. LaToza, G. Venolia, and R. DeLine, "Maintaining mental models: a study of developer work habits," in *Proceedings of the 28th international conference on Software engineering*, 2006, pp. 492–501.
- [6] E. Aghajani, C. Nagy, M. Linares-Vásquez, L. Moreno, G. Bavota, M. Lanza, and D. C. Shepherd, "Software documentation: the practitioners' perspective," in *Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering*, 2020, pp. 590–601.
- [7] J. Börstler and B. Paech, "The role of method chains and comments in software readability and comprehension—an experiment," *IEEE Transactions on Software Engineering*, vol. 42, no. 9, pp. 886–898, 2016.
- [8] S. Stapleton, Y. Gambhir, A. LeClair, Z. Eberhart, W. Weimer, K. Leach, and Y. Huang, "A human study of comprehension and code summarization," in *Proceedings of the 28th International Conference on Program Comprehension*, 2020, pp. 2–13.
- [9] T. Tenny, "Program readability: Procedures versus comments," *IEEE Transactions on Software Engineering*, vol. 14, no. 9, p. 1271, 1988.
- [10] S. N. Woodfield, H. E. Dunsmore, and V. Y. Shen, "The effect of modularization and comments on program comprehension," in *Proceedings of the 5th international conference on Software engineering*, 1981, pp. 215–223.
- [11] B. P. Lientz, "Issues in software maintenance," *ACM Computing Surveys (CSUR)*, vol. 15, no. 3, pp. 271–278, 1983.
- [12] N. J. Abid, N. Dragan, M. L. Collard, and J. I. Maletic, "Using stereotypes in the automatic generation of natural language summaries for c++ methods," in *2015 IEEE International Conference on Software Maintenance and Evolution (ICSME)*. IEEE, 2015, pp. 561–565.
- [13] W. U. Ahmad, S. Chakraborty, B. Ray, and K.-W. Chang, "Unified pre-training for program understanding and generation," *arXiv preprint arXiv:2103.06333*, 2021.
- [14] M. Allamanis, H. Peng, and C. Sutton, "A convolutional attention network for extreme summarization of source code," in *International conference on machine learning*. PMLR, 2016, pp. 2091–2100.
- [15] A. V. M. Barone and R. Sennrich, "A parallel corpus of python functions and documentation strings for automated code documentation and code generation," *arXiv preprint arXiv:1707.02275*, 2017.
- [16] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "Bert: Pre-training of deep bidirectional transformers for language understanding," *arXiv preprint arXiv:1810.04805*, 2018.
- [17] B. P. Eddy, J. A. Robinson, N. A. Kraft, and J. C. Carver, "Evaluating source code summarization techniques: Replication and expansion," in *2013 21st International Conference on Program Comprehension (ICPC)*. IEEE, 2013, pp. 13–22.
- [18] Z. Feng, D. Guo, D. Tang, N. Duan, X. Feng, M. Gong, L. Shou, B. Qin, T. Liu, D. Jiang *et al.*, "Codebert: A pre-trained model for programming and natural languages," *arXiv preprint arXiv:2002.08155*, 2020.
- [19] S. Gao, C. Gao, Y. He, J. Zeng, L. Nie, X. Xia, and M. Lyu, "Code structure-guided transformer for source code summarization," *ACM Transactions on Software Engineering and Methodology*, vol. 32, no. 1, pp. 1–32, 2023.
- [20] S. Haiduc, J. Aponte, and A. Marcus, "Supporting program comprehension with source code summarization," in *Proceedings of the 32nd ACM/IEEE International Conference on Software Engineering-Volume 2*, 2010, pp. 223–226.
- [21] S. Haiduc, J. Aponte, L. Moreno, and A. Marcus, "On the use of automated text summarization techniques for summarizing source code," in *2010 17th Working Conference on Reverse Engineering*. IEEE, 2010, pp. 35–44.
- [22] X. Hu, G. Li, X. Xia, D. Lo, and Z. Jin, "Deep code comment generation," in *Proceedings of the 26th conference on program comprehension*, 2018, pp. 200–210.
- [23] —, "Deep code comment generation with hybrid lexical and syntactical information," *Empirical Software Engineering*, vol. 25, pp. 2179–2217, 2020.
- [24] S. Iyer, I. Konstas, A. Cheung, and L. Zettlemoyer, "Summarizing source code using a neural attention model," in *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2016, pp. 2073–2083.
- [25] J. Y. Khan and G. Uddin, "Automatic code documentation generation using gpt-3," in *37th IEEE/ACM International Conference on Automated Software Engineering*, 2022, pp. 1–6.
- [26] S. Liu, Y. Chen, X. Xie, J. Siow, and Y. Liu, "Retrieval-augmented generation for code summarization via hybrid gnn," *arXiv preprint arXiv:2006.05405*, 2020.
- [27] P. W. McBurney and C. McMillan, "Automatic documentation generation via source code summarization of method context," in *Proceedings of the 22nd International Conference on Program Comprehension*, 2014, pp. 279–290.
- [28] L. Moreno, J. Aponte, G. Sridhara, A. Marcus, L. Pollock, and K. Vijay-Shanker, "Automatic generation of natural language summaries for java classes," in *2013 21st International Conference on Program Comprehension (ICPC)*. IEEE, 2013, pp. 23–32.
- [29] M. R. Parvez, W. U. Ahmad, S. Chakraborty, B. Ray, and K.-W. Chang, "Retrieval augmented code generation and summarization," *arXiv preprint arXiv:2108.11601*, 2021.
- [30] L. Phan, H. Tran, D. Le, H. Nguyen, J. Anibal, A. Peltekian, and Y. Ye, "Cotext: Multi-task learning with code-text transformer," *arXiv preprint arXiv:2105.08645*, 2021.
- [31] S. Rastkar, G. C. Murphy, and A. W. Bradley, "Generating natural language summaries for crosscutting source code concerns," in *2011 27th IEEE International Conference on Software Maintenance (ICSM)*. IEEE, 2011, pp. 103–112.
- [32] P. Rodeghero, C. McMillan, P. W. McBurney, N. Bosch, and S. D'Mello, "Improving automated source code summarization via an eye-tracking study of programmers," in *Proceedings of the 36th international conference on Software engineering*, 2014, pp. 390–401.
- [33] G. Sridhara, E. Hill, D. Muppaneni, L. Pollock, and K. Vijay-Shanker, "Towards automatically generating summary comments for java methods," in *Proceedings of the IEEE/ACM international conference on Automated software engineering*, 2010, pp. 43–52.
- [34] Y. Wan, Z. Zhao, M. Yang, G. Xu, H. Ying, J. Wu, and P. S. Yu, "Improving automatic source code summarization via deep reinforcement learning," in *Proceedings of the 33rd ACM/IEEE international conference on automated software engineering*, 2018, pp. 397–407.
- [35] Y. Wang, W. Wang, S. Joty, and S. C. Hoi, "Codet5: Identifier-aware unified pre-trained encoder-decoder models for code understanding and generation," *arXiv preprint arXiv:2109.00859*, 2021.
- [36] E. Wong, J. Yang, and L. Tan, "Autocomment: Mining question and answer sites for automatic comment generation," in *2013 28th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 2013, pp. 562–567.
- [37] E. Wong, T. Liu, and L. Tan, "Clocom: Mining existing source code for automatic comment generation," in *2015 IEEE 22nd International Conference on Software Analysis, Evolution, and Reengineering (SANER)*. IEEE, 2015, pp. 380–389.
- [38] Q. Chen, X. Xia, H. Hu, D. Lo, and S. Li, "Why my code summarization model does not work: Code comment improvement with category prediction," *ACM Transactions on Software Engineering and Methodology (TOSEM)*, vol. 30, no. 2, pp. 1–29, 2021.
- [39] M. Geng, S. Wang, D. Dong, H. Wang, G. Li, Z. Jin, X. Mao, and X. Liao, "Large language models are few-shot summarizers: Multi-intent comment generation via in-context learning," in *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering*, 2024, pp. 1–13.
- [40] X. Song, H. Sun, X. Wang, and J. Yan, "A survey of automatic generation of source code comments: Algorithms and techniques," *IEEE Access*, vol. 7, pp. 111 411–111 428, 2019.
- [41] J. Wang, Y. Liang, F. Meng, Z. Sun, H. Shi, Z. Li, J. Xu, J. Qu, and J. Zhou, "Is chatgpt a good nlg evaluator? a preliminary study," *arXiv preprint arXiv:2303.04048*, 2023.
- [42] D. Steidl, B. Hummel, and E. Juergens, "Quality analysis of source code comments," in *2013 21st International Conference on Program Comprehension (ICPC)*, 2013, pp. 83–92.
- [43] N. Khamis, R. Witte, and J. Rilling, "Automatic quality assessment of source code comments: the javadocminer," in *International Conference on Application of Natural Language to Information Systems*. Springer, 2010, pp. 68–79.
- [44] R. P. Buse and W. R. Weimer, "Automatically documenting program changes," in *Proceedings of the IEEE/ACM international conference on Automated software engineering*, 2010, pp. 33–42.

- [45] —, “Automatic documentation inference for exceptions,” in *Proceedings of the 2008 international symposium on Software testing and analysis*, 2008, pp. 273–282.
- [46] C. Raffel, N. Shazeer, A. Roberts, K. Lee, S. Narang, M. Matena, Y. Zhou, W. Li, and P. J. Liu, “Exploring the limits of transfer learning with a unified text-to-text transformer,” *The Journal of Machine Learning Research*, vol. 21, no. 1, pp. 5485–5551, 2020.
- [47] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, I. Sutskever *et al.*, “Language models are unsupervised multitask learners,” *OpenAI blog*, vol. 1, no. 8, p. 9, 2019.
- [48] H. Husain, H.-H. Wu, T. Gazit, M. Allamanis, and M. Brockschmidt, “Codesearchnet challenge: Evaluating the state of semantic code search,” *arXiv preprint arXiv:1909.09436*, 2019.
- [49] P. Ellingwood, H. Enxing, L. Hamilton, J. Harer, T. Jost, L. Kim, L. Kosta, M. McConley, O. Ozdemir, C. Reale *et al.*, “Draper mining and understanding software enclaves (muse),” 2019.
- [50] Chris Lattner, LLVM Developer Group, “Clang.” [Online]. Available: <https://clang.llvm.org/>
- [51] M. M. Hennink, B. N. Kaiser, and V. C. Marconi, “Code saturation versus meaning saturation: how many interviews are enough?” *Qualitative health research*, vol. 27, no. 4, pp. 591–608, 2017.
- [52] A. S. Miner, “Saturation for a general class of models,” *IEEE Transactions on Software Engineering*, vol. 32, no. 8, pp. 559–570, 2006.
- [53] J. Sillito, G. C. Murphy, and K. De Volder, “Questions programmers ask during software evolution tasks,” in *Proceedings of the 14th ACM SIGSOFT international symposium on Foundations of software engineering*, 2006, pp. 23–34.
- [54] K. L. Gwet, “On the krippendorff’s alpha coefficient,” *Manuscript submitted for publication. Retrieved October*, vol. 2, no. 2011, p. 2011, 2011.
- [55] J. L. Fleiss, “Measuring nominal scale agreement among many raters.” *Psychological bulletin*, vol. 76, no. 5, p. 378, 1971.
- [56] X. Hu, X. Xia, D. Lo, Z. Wan, Q. Chen, and T. Zimmermann, “Practitioners’ expectations on automated code comment generation,” in *Proceedings of the 44th International Conference on Software Engineering*, 2022, pp. 1693–1705.
- [57] J. White, Q. Fu, S. Hays, M. Sandborn, C. Olea, H. Gilbert, A. Elnashar, J. Spencer-Smith, and D. C. Schmidt, “A prompt pattern catalog to enhance prompt engineering with chatgpt,” *Proceedings of the 30th Conference on Pattern Languages of Programs*, 2023.
- [58] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, E. Kaiser, and I. Polosukhin, “Attention is all you need,” *Advances in neural information processing systems*, vol. 30, 2017.
- [59] Y. Liu, M. Ott, N. Goyal, J. Du, M. Joshi, D. Chen, O. Levy, M. Lewis, L. Zettlemoyer, and V. Stoyanov, “Roberta: A robustly optimized bert pretraining approach,” *arXiv preprint arXiv:1907.11692*, 2019.
- [60] A. Farahani, S. Voghoei, K. Rasheed, and H. R. Arabnia, “A brief review of domain adaptation,” *Advances in data science and information engineering: proceedings from ICDATA 2020 and IKE 2020*, pp. 877–894, 2021.
- [61] OpenAI, “Chatgpt.” [Online]. Available: <https://chat.openai.com/>
- [62] W. Medhat, A. Hassan, and H. Korashy, “Sentiment analysis algorithms and applications: A survey,” *Ain Shams engineering journal*, vol. 5, no. 4, pp. 1093–1113, 2014.
- [63] J. C. D. Terry, H. E. Roy, and T. A. August, “Thinking like a naturalist: Enhancing computer vision of citizen science images by harnessing contextual data,” *Methods in Ecology and Evolution*, vol. 11, no. 2, pp. 303–315, 2020.
- [64] GitHub, OpenAI, “Github copilot.” [Online]. Available: <https://github.com/features/copilot>
- [65] R. Plösch, A. Dautovic, and M. Saft, “The value of software documentation quality,” in *2014 14th International Conference on Quality Software*. IEEE, 2014, pp. 333–342.
- [66] X. Xia, L. Bao, D. Lo, Z. Xing, A. E. Hassan, and S. Li, “Measuring program comprehension: A large-scale field study with professionals,” *IEEE Transactions on Software Engineering*, vol. 44, no. 10, pp. 951–976, 2017.
- [67] Y. Bang, S. Cahyawijaya, N. Lee, W. Dai, D. Su, B. Wilie, H. Lovenia, Z. Ji, T. Yu, W. Chung *et al.*, “A multitask, multilingual, multimodal evaluation of chatgpt on reasoning, hallucination, and interactivity,” *arXiv preprint arXiv:2302.04023*, 2023.